Shell From Scratch

A custom Unix-like shell built from scratch in **C/C++** to understand process management, system calls, file descriptors, inter-process communication, signals, job control, parsing, and terminal interaction.

The project is being developed by a **3-person team**. Each member owns a major subsystem, while integration, testing, debugging, and architectural decisions are handled collaboratively.

---

1. Project Goals

The shell should progressively support:

- Interactive command execution
- Built-in commands
- Command parsing and tokenization
- Environment variables
- Input/output redirection
- Pipes
- Background processes
- Signal handling
- Job control
- Command history
- Tab completion
- Error handling
- Exit-status handling

The main objective isn't just to make a command prompt that works. The team should implement the underlying OS interactions directly so that the project demonstrates how a real shell communicates with the operating system.

---

2. Technology Stack

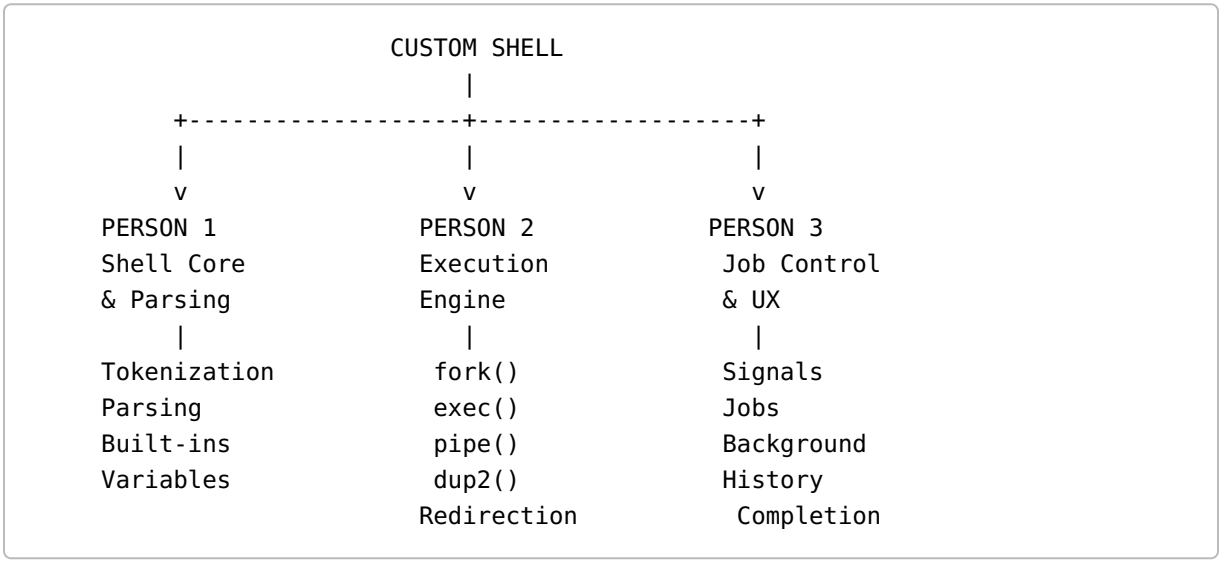
Component	Technology
Language	C/C++
Target OS	Linux
Build System	Make / CMake
Version Control	Git + GitHub
Debugging	GDB

Component	Technology
System-call inspection	`strace`
Compiler	GCC / G++
Testing	Custom test scripts + manual testing

---

3. Team Structure

The project is split into three major areas:



Person 1 — Shell Core & Parser

Owns everything related to understanding and structuring user input:

- Input handling
- Tokenization
- Parsing
- Command representation
- Built-in commands
- Environment variables

Person 2 — Process Execution & I/O

Owns everything related to actually executing commands:

- `fork()`
- `exec()`

- `waitpid()`
- Pipes
- Redirection
- File descriptors
- Process synchronization

Person 3 — Job Control, Signals & User Experience

Owns advanced shell behavior:

- Background processes
- Signals
- Job control
- Process groups
- Command history
- Tab completion
- Terminal interaction

The three areas are intentionally connected. Person 1 produces the command structure that Person 2 executes, while Person 3 manages the processes and terminal behavior around that execution.

---

4. Person 1 — Shell Core & Parser

Main Responsibility

Person 1 is responsible for turning raw user input into a structured representation that the execution engine can understand.

The pipeline is:

```
User Input
    ↓
Input Handler
    ↓
Tokenizer
    ↓
Parser
    ↓
Command / Pipeline Structure
    ↓
Executor
```

4.1 Input Handling

Implement the interactive shell loop:

```
Display prompt
      ↓
Read user input
      ↓
Tokenize
      ↓
Parse
      ↓
Create command structure
      ↓
Pass to executor
```

Example:

```
myshell> ls -la
```

4.2 Tokenization

Convert:

```
cat file.txt | grep hello > result.txt
```

into tokens such as:

```
cat
file.txt
|
grep
hello
>
result.txt
```

The tokenizer should recognize:

```
WORD
|
```

```
>
>>
<
&
;
```

Eventually, quoted strings should also be supported:

```
echo "hello world"
```

The tokenizer must treat `"hello world"` as one logical argument.

4.3 Parser

Convert the token stream into an internal representation.

A possible structure:

```
struct Command {
    std::vector<std::string> args;

    std::string inputFile;
    std::string outputFile;

    bool append;
};

struct Pipeline {
    std::vector<Command> commands;
    bool background;
};
```

For:

```
cat file.txt | grep hello > result.txt
```

the parser should produce a structure similar to:

```
Pipeline
|
├── Command
```

```
│   ├── cat
│   └── file.txt
│
└── Command
    ├── grep
    ├── hello
    └── output → result.txt
```

This structure becomes the main interface between **Person 1 and Person 2**.

4.4 Built-in Commands

Implement built-ins such as:

```
cd
pwd
echo
exit
export
unset
```

An important example is `cd`.

`cd` must execute inside the shell process because changing the directory of a child process doesn't change the parent shell's working directory.

4.5 Environment Variables

Support:

```
export NAME=value
echo $NAME
unset NAME
```

Person 1 Deliverables

- [ ] Input loop
- [ ] Tokenizer
- [ ] Parser

- [ ] Command data structures
- [ ] Built-in commands
- [ ] Environment-variable handling
- [ ] Parser error handling
- [ ] Parser unit tests

---

5. Person 2 — Process Execution & I/O

Person 2 owns the execution engine.

The main responsibility is to take the structure produced by Person 1 and turn it into actual Linux processes.

5.1 Process Creation

Understand and implement:

```
fork()
exec()
wait()
waitpid()
```

For:

```
ls -la
```

the execution flow should conceptually be:

```
            Shell
              |
            fork()
          /
         /
     Parent          Child
       |               |
   waitpid()       exec()
                      |
                      v
                     ls
```

5.2 Command Execution

Implement commands such as:

```
ls
pwd
cat file.txt
grep hello file.txt
```

Use the appropriate `exec*()` functions instead of `system()`.

The goal is to understand how the shell creates and manages processes rather than hiding that work behind a high-level API.

---

6. Redirection

Implement:

Output redirection

```
ls > output.txt
```

Append

```
ls >> output.txt
```

Input redirection

```
sort < input.txt
```

Optional stderr redirection

```
command 2> error.txt
```

Important system calls:

```
open()
close()
dup2()
```

The team should understand the standard file descriptors:

```
0 → stdin
1 → stdout
2 → stderr
```

---

7. Pipes

Implement:

```
ls | grep cpp
```

Conceptually:

```
       stdout              stdin
          |                  |
          v                  v
        +-----+            +------+
        | PIPE| ---------->| grep |
        +-----+            +------+
           ^
           |
         ls
```

The implementation will involve:

```
pipe()
fork()
dup2()
close()
exec()
```

7.1 Multiple Pipes

Eventually support:

```
cat file.txt | grep hello | sort | uniq
```

The execution engine must correctly create multiple pipes and connect the correct input/output descriptors for every process.

7.2 Process Synchronization

Implement:

```
wait()
waitpid()
```

and correctly handle:

- Child termination
- Exit status
- Multiple processes in a pipeline
- Background processes
- Cleanup of child processes

Person 2 Deliverables

- [ ] Process creation
- [ ] Command execution
- [ ] `fork()`
- [ ] `exec()`
- [ ] `waitpid()`
- [ ] Input redirection
- [ ] Output redirection
- [ ] Append redirection
- [ ] Pipes
- [ ] Multiple pipelines
- [ ] Exit-status handling
- [ ] Execution error handling

---

8. Person 3 — Job Control, Signals & User Experience

Person 3 handles the features that make the shell behave like an interactive Unix shell rather than simply executing commands.

---

9. Background Processes

Support:

```
sleep 10 &
```

Expected behavior:

```
myshell> sleep 10 &
[1] 12345

myshell>
```

The shell should immediately return to the prompt instead of waiting for the process to finish.

---

10. Signal Handling

Handle important signals:

```
SIGINT
SIGTSTP
SIGCHLD
SIGCONT
SIGTERM
SIGQUIT
```

Important terminal interactions:

```
Ctrl+C → SIGINT
Ctrl+Z → SIGTSTP
```

The shell itself should remain running when a foreground program is interrupted.

Conceptually:

```
myshell
   |
   +---- foreground process
              |
           Ctrl+C
              |
           SIGINT
              |
         process exits
              |
              v
           myshell
```

For robust signal handling, the team should prefer understanding and using `sigaction()` rather than relying only on the older `signal()` interface.

---

11. Job Control

Implement:

```
jobs
fg
bg
```

Example:

```
myshell> sleep 100 &
[1] 1234

myshell> jobs
[1] Running    sleep 100

myshell> fg 1
```

Advanced concepts include:

- Process IDs

- Process groups
- Foreground process group
- Background process group
- Terminal ownership
- `setpgid()`
- `tcsetpgrp()`

This is one of the most OS-intensive parts of the project.

---

12. Command History

Implement:

```
history
```

and:

```
↑
↓
```

Example:

```
myshell> ls
myshell> pwd
myshell> cd Documents
myshell> ↑
```

The shell should retrieve previously entered commands.

Optional persistent history:

```
~/.myshell_history
```

---

13. Tab Completion

Support basic completion:

```
myshell> cd Doc<TAB>
```

becoming:

```
myshell> cd Documents/
```

Completion can initially support:

- Files
- Directories
- Executable commands

If the project is specifically intended to teach terminal handling, implement this yourself. Otherwise, a library such as `readline` can be considered later.

Person 3 Deliverables

- [ ] Background processes
- [ ] Signal handling
- [ ] `jobs`
- [ ] `fg`
- [ ] `bg`
- [ ] Process groups
- [ ] Foreground/background control
- [ ] Command history
- [ ] Arrow-key navigation
- [ ] Tab completion
- [ ] Interactive terminal handling

---

14. Shared Responsibilities

The subsystem ownership is clear, but the project should still be treated as **one codebase**.

Everyone contributes to:

- Integration
- Debugging
- Code reviews
- Documentation
- Testing
- Architecture decisions

- Final demonstrations

---

15. Git Workflow

Each member should work on a feature branch.

Example:

```
main
 |
 +-- feature/parser
 |
 +-- feature/executor
 |
 +-- feature/job-control
```

Recommended workflow:

```
Create branch
      ↓
Implement feature
      ↓
Test locally
      ↓
Commit
      ↓
Push
      ↓
Pull Request
      ↓
Code Review
      ↓
Merge
      ↓
Integration Test
```

Example commit messages:

```
feat: implement command tokenizer
feat: add output redirection
feat: implement pipeline execution
feat: add SIGINT handling
```

```
fix: close unused pipe descriptors
test: add parser edge cases
```

---

16. Shared Coding Rules

Use consistent naming.

Prefer:

```
parseCommand()
executeCommand()
handleRedirection()
createPipeline()
handleSignal()
addJob()
```

over:

```
abc()
xyz()
func1()
test()
```

Keep functions focused and avoid putting the entire shell inside `main.cpp`.

A good rule is:

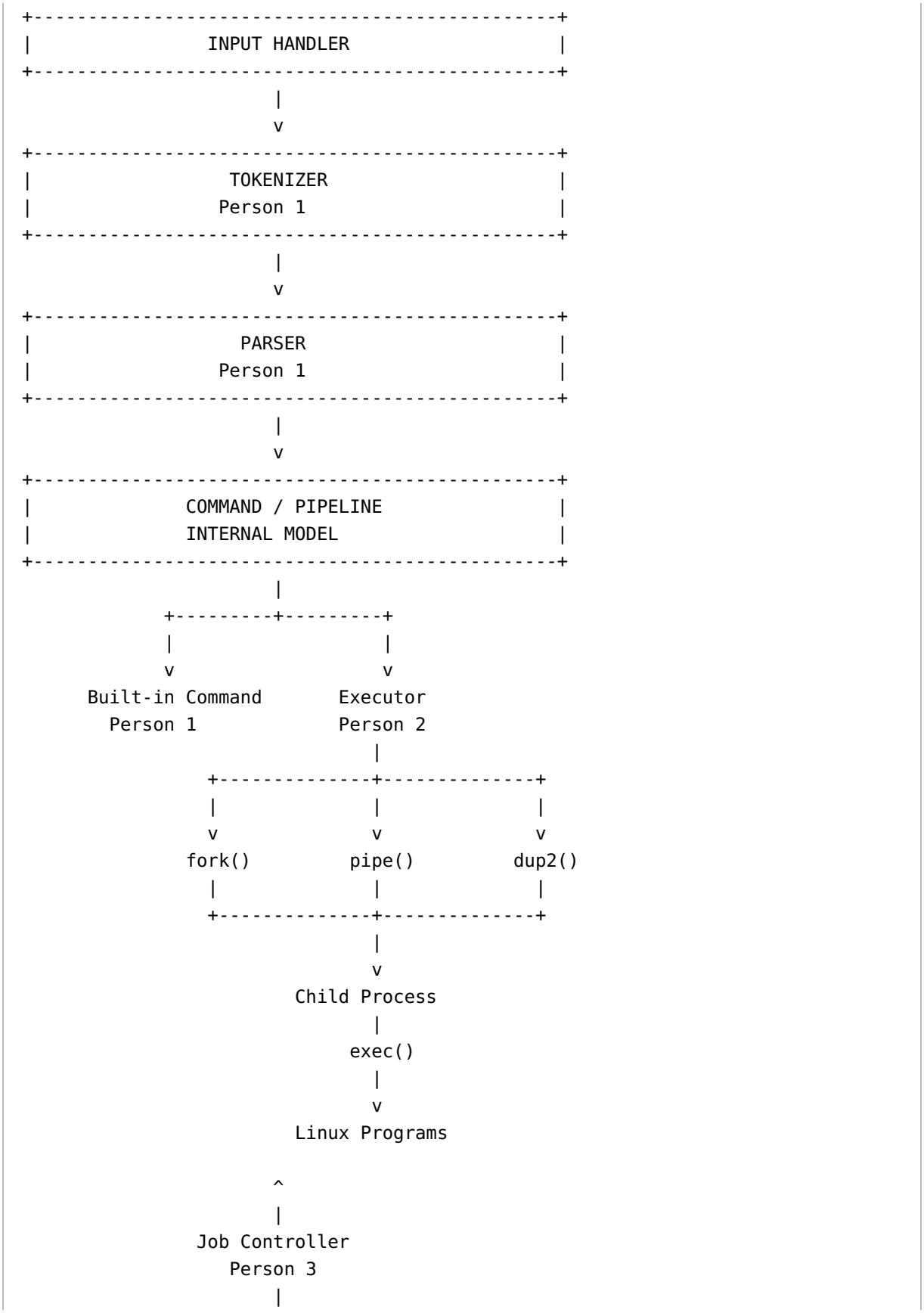
> If one function is doing input parsing, process creation, redirection, signal handling, and job management at the same time, it's probably time to split it.

---

17. Project Architecture

The final architecture should approximately look like:

```
+-----------------------------------------------+
|                  USER INPUT                   |
+-----------------------------------------------+
                       |
                       v
```


INPUT HANDLER
TOKENIZER
Person 1
PARSER
Person 1
COMMAND / PIPELINE
INTERNAL MODEL
Built-in Command
Person 1
Executor
Person 2
fork()
pipe()
dup2()
Child Process
exec()
Linux Programs
Job Controller
Person 3

```
            +---------+---------+
            |         |         |
          Signals   Jobs      Terminal
```

---

18. Repository Structure

A practical starting structure is:

```
myshell/
│
├── src/
│   ├── main.cpp
│   │
│   ├── parser/
│   │   ├── tokenizer.cpp
│   │   ├── tokenizer.h
│   │   ├── parser.cpp
│   │   └── parser.h
│   │
│   ├── builtins/
│   │   ├── builtins.cpp
│   │   └── builtins.h
│   │
│   ├── executor/
│   │   ├── executor.cpp
│   │   ├── executor.h
│   │   ├── process.cpp
│   │   └── process.h
│   │
│   ├── jobs/
│   │   ├── jobs.cpp
│   │   ├── jobs.h
│   │   ├── signals.cpp
│   │   └── signals.h
│   │
│   └── terminal/
│       ├── history.cpp
│       ├── history.h
│       ├── completion.cpp
│       └── completion.h
│
├── tests/
│   ├── parser/
│   ├── executor/
```

```
│   ├── pipes/
│   ├── redirection/
│   └── jobs/
│
├── docs/
│
├── Makefile
├── README.md
└── .gitignore
```

---

19. Development Roadmap

Phase 0 — Prerequisites

Duration: 3–5 days

All three members should understand:

- C/C++
- Pointers
- Structures/classes
- Linux commands
- Processes
- File descriptors
- System calls
- Git
- Basic debugging

Important system calls:

```
fork()
exec()
wait()
waitpid()
pipe()
dup2()
open()
close()
read()
write()
chdir()
```

```
getcwd()
kill()
```

The team should also understand the relationship between:

```
Shell
 ↓
System Calls
 ↓
Linux Kernel
 ↓
Hardware
```

Phase 1 — Minimal Shell

Duration: Week 1

Person 1

- Input loop
- Basic tokenizer
- Basic command representation
- Basic built-ins

Person 2

- `fork()`
- `exec()`
- `waitpid()`
- Basic command execution

Person 3

- Project structure
- Makefile
- Testing framework
- Documentation
- Basic error handling

Target

```
myshell> ls
myshell> pwd
```

```
myshell> echo hello
myshell> cd ..
myshell> exit
```

Phase 2 — Parser & Redirection

Duration: Week 2

Person 1

- Improved tokenizer
- Operators
- Quotes
- Parser
- Command structures

Person 2

- `<`
- `>`
- `>>`
- File descriptors
- `dup2()`

Person 3

- Parser tests
- Redirection tests
- Error cases
- Integration testing

Target

```
cat input.txt
cat < input.txt
ls > output.txt
ls >> output.txt
```

Phase 3 — Pipes

Duration: Week 3

Person 1

- Pipeline parsing
- Multiple command structures

Person 2

- `pipe()`
- Pipe creation
- `dup2()`
- Multiple pipelines
- Process synchronization

Person 3

- Pipeline testing
- Resource-leak testing
- Error handling

Target

```
ls | grep cpp
cat file.txt | grep hello | sort
```

Phase 4 — Background Processes & Signals

Duration: Week 4

Person 1

- Background operator parsing
- Built-in improvements

Person 2

- Background process execution
- Child-process tracking

Person 3

- Signal handling
- `SIGINT`
- `SIGTSTP`
- `SIGCHLD`
- `SIGCONT`

Target

```
sleep 10 &
```

and:

```
Ctrl+C
Ctrl+Z
```

should behave correctly.

Phase 5 — Job Control

Duration: Week 5

Person 1

- Job-related command parsing
- `jobs` built-in integration

Person 2

- Process groups
- Process tracking
- Exit status

Person 3

- `jobs`
- `fg`
- `bg`
- Foreground/background control
- Terminal ownership

Target

```
sleep 100 &
jobs
fg 1
Ctrl+Z
bg 1
```

Phase 6 — Shell UX

Duration: Week 6

Person 1

- Environment variables
- Aliases
- Built-in improvements

Person 2

- Execution error handling
- Exit-status improvements
- Resource cleanup

Person 3

- Command history
- Arrow navigation
- Tab completion
- Prompt customization

Target

A usable interactive shell that supports both core Unix behavior and a reasonable interactive experience.

---

20. Testing Strategy

Every feature should be tested with:

Normal case

```
ls
```

Edge case

```
ls |
```

Invalid command

```
this_command_does_not_exist
```

Empty input

```
myshell>
```

Multiple commands

```
ls | grep cpp | sort
```

Resource cleanup

Check that file descriptors and child processes aren't leaked.

---

21. Important Test Cases

The final shell should be tested against at least:

```
pwd

ls

ls -la

echo hello

cd ..

cat file.txt

cat < input.txt

ls > output.txt

ls >> output.txt

ls | grep cpp

cat file.txt | grep hello | sort
```

```
sleep 5 &

jobs

fg 1

bg 1

Ctrl+C

Ctrl+Z

export NAME=value

echo $NAME

unset NAME

history
```

---

22. What NOT to Do

Don't use `system()` as the main execution mechanism

Avoid:

```
system(command.c_str());
```

That hides the process-management work this project is supposed to teach.

Instead, understand and implement:

```
fork()
   ↓
exec()
   ↓
waitpid()
```

Don't make one giant `main.cpp`

Avoid:

```
main.cpp
  └── 1500 lines
```

Separate the project into logical modules.

Don't wait until the end to integrate

Integration should happen continuously.

Use this cycle:

```
Implement
   ↓
Test
   ↓
Commit
   ↓
Pull Request
   ↓
Code Review
   ↓
Merge
   ↓
Integration Test
```

---

23. Definition of Done

The project can be considered complete when:

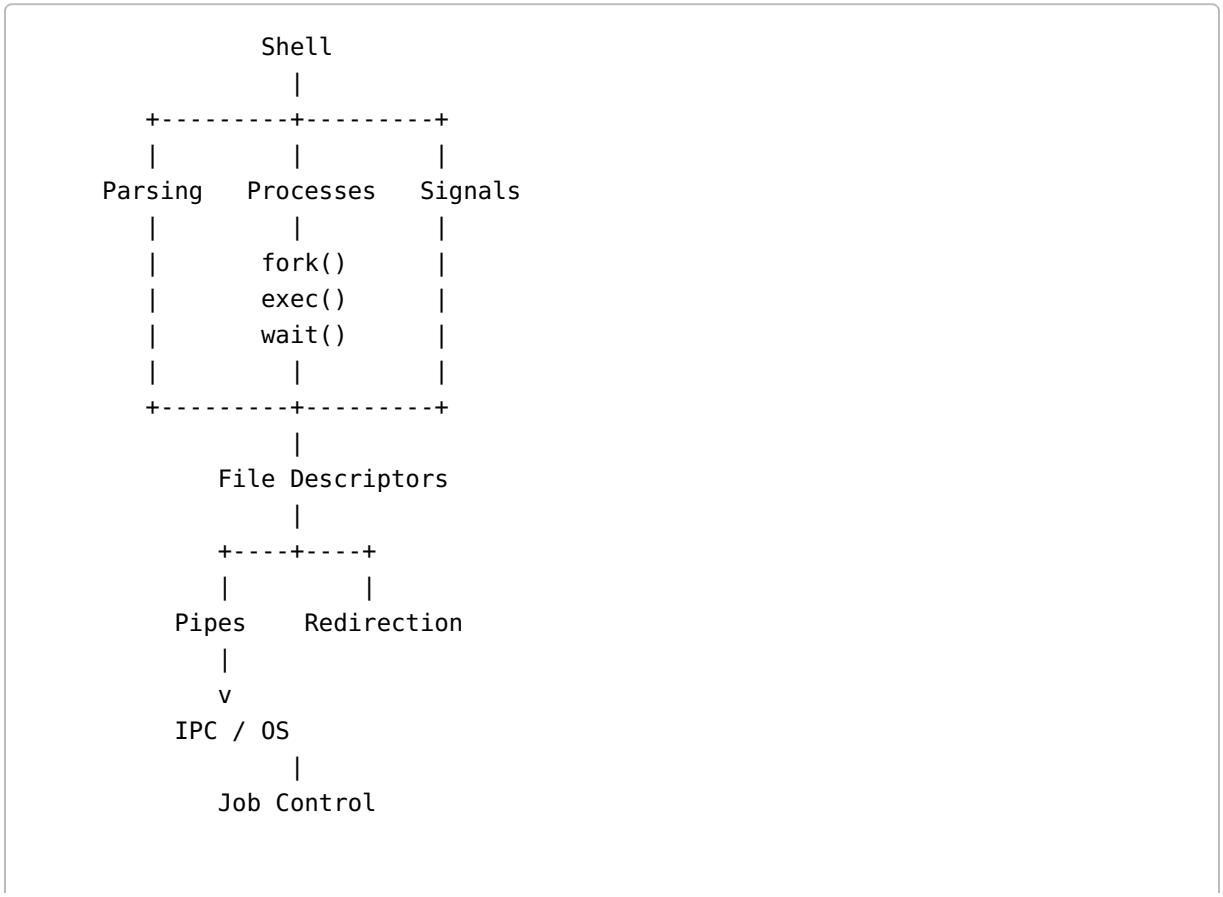
- [ ] Shell starts successfully
- [ ] Commands can be executed
- [ ] Built-ins work
- [ ] Arguments are parsed correctly
- [ ] Quotes are handled
- [ ] Input redirection works
- [ ] Output redirection works

- [ ] Append redirection works
- [ ] Pipes work
- [ ] Multiple pipes work
- [ ] Background processes work
- [ ] Signals are handled
- [ ] Jobs can be listed
- [ ] Jobs can be moved between foreground/background
- [ ] Environment variables work
- [ ] Command history works
- [ ] Basic tab completion works
- [ ] Invalid commands are handled gracefully
- [ ] Child processes are cleaned up
- [ ] File descriptors are properly closed
- [ ] Code is modular
- [ ] Documentation is complete
- [ ] All team members have contributed through Git

---

24. Final Learning Outcomes

By completing this project, the team should understand:



```
                |
            Terminal
```

The project shouldn't be judged only by whether the final shell can run `ls` and `cd`.

The real outcome is that the team can explain **what happens from the moment a user enters a command until the corresponding process runs**, including parsing, `fork()`, `exec()`, file descriptors, pipes, signals, process groups, and terminal control.

That makes the project a strong practical introduction to operating-system internals rather than just a command-line UI.